LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – RELASI ANTAR KELAS

NAMA : Elok Elysia Dewi

NIM : 103132400013

KELAS : S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. **Diagram Kelas**
   Diagram kelas adalah diagram yang digunakan untuk memodelkan kelas-kelas dalam sistem beserta hubungan antar kelasnya. Kelas digambarkan dengan kotak yang berisi nama kelas, atribut, dan method. Simbol access modifier: + (public), - (private), dan # (protected).

2. **Asosiasi**
   Asosiasi adalah hubungan antara dua kelas terpisah yang terbentuk melalui objek pada tiap kelas. Method milik satu kelas dipanggil di kelas lain, tetapi objeknya tidak menjadi atribut kelas tersebut. Digambarkan dengan garis lurus.

3. **Agregasi**
   Agregasi adalah hubungan yang lebih ketat antara dua kelas, yaitu hubungan bagian (part) dan keseluruhan (whole). Kelas part dan whole dapat diciptakan sendiri-sendiri. Digambarkan dengan garis berujung belah ketupat kosong.

4. **Komposisi**
   Komposisi adalah bentuk khusus dari agregasi dengan hubungan yang lebih kuat, karena part tidak akan ada tanpa adanya kelas whole. Jika whole musnah, part ikut musnah. Digambarkan dengan garis berujung belah ketupat hitam.

5. **Generalisasi dan Spesialisasi**
   Hubungan yang terkait dengan pewarisan, yaitu suatu kelas merupakan turunan dari kelas lain. Digambarkan dengan garis berujung segitiga kosong mengarah ke kelas induk.

B. Guided

1. Dosen (Asosiasi)

```
package Guided;


public class Dosen {
    private String kodeDosen;
    private String namaDosen;


    //Setter
```

```
public void setKodeDosen(String kodeDosen) {
    this.kodeDosen = kodeDosen;
}

public void setNamaDosen(String namaDosen) {
    this.namaDosen = namaDosen;
}

//Getter
public String getKodeDosen() {
    return this.kodeDosen;
}

public String getNamaDosen() {
    return this.namaDosen;
}

public void giveScore(Mahasiswa s, int nilai) {
     s.setNilai(nilai);
}

public int getScore(Mahasiswa s) {
     return s.getNilai();
}

public static void main(String[] args) {
```

```
        Mahasiswa m = new Mahasiswa();


        m.setNim("130118383");
        m.setNama("Budi");


        Dosen d = new Dosen();
        d.giveScore(m, 90);


        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Mahasiswa

```
package Guided;



public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;


    //Setter
    public void setNama(String nama) {
        this.nama = nama;
    }
```

```
    public void setNim(String nim) {
        this.nim = nim;
    }

    public String getNama() {
        return this.nama;
    }

    public void setNilai(int nilai){
        this.nilai = nilai;
    }

    //Getter
    public int getNilai(){
        return this.nilai;
    }


    public String getNim() {
        return this.nim;
    }
}
```

Output :

```
Nim :130118383
Nama :Budi
Nilai :90
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  0.983 s
Finished at: 2026-10-19T19:00:54+07:00
```

2. Jurusan (Agregasi)

```
package Guided.Agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan (String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //Behaviour
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i=0; i<jumlahMahasiswa;i++){
            System.out.println(i+1 + "," +
            daftarMahasiswa[i].getNIM() + " "
            + daftarMahasiswa[i].gerNama());
        }
    }

    //Getter
```

```
    public String getKodeJurusan(){
        return kodeJurusan;
    }

    public String getNamaJurusan(){
        return namaJurusan;
    }
    //Setter

    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }
}
```

Mahasiswa

```
package Guided.Agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String getNIM(){
        return NIM;
    }
    public String gerNama(){
        return Nama;
    }

    //setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
```

```
    }
}
```

Main

```
package Guided.Agregasi;

public class Main {
    public static void main(String[] args) {
        Jurusan j = new Jurusan("IF", "Informatika");

        Mahasiswa m1 = new Mahasiswa("2024001", "Budi");
        Mahasiswa m2 = new Mahasiswa("2024002", "Siti");

        j.addMahasiswa(m1);
        j.addMahasiswa(m2);

        j.displayMahasiswa();
    }
}
```

Output :

```
Kode Jurusan: IF
Nama Jurusan: Informatika
Daftar Mahasiswa:
1,2024001 Budi
2,2024002 Siti
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  0.979 s
Finished at: 2026-10-19T19:42:56+07:00
```

3. Processor (Komposisi)

```
package Guided.Komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
```

```
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Monitor

```
package Guided.Komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

Mouse

```
package Guided.Komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

Komputer

```
package Guided.Komposisi;

public class Komputer {
    private String merk;
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String merk, String brandProcessor, int kecepatan,
                    String brandMonitor, String brandMouse, String tipeMouse) {
        this.merk = merk;
        // objek dibuat DI DALAM class Komputer (ciri komposisi)
        this.processor = new Processor(brandProcessor, kecepatan);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpec() {
        System.out.println("Komputer: " + this.merk);
        System.out.print("Processor: ");
        processor.displaySpec();
        System.out.print("Monitor: ");
        monitor.DisplaySpec();
        System.out.print("Mouse: ");
        mouse.displaySpec();
    }
}
```

Main

```
package Guided.Komposisi;

public class Main {
    public static void main(String[] args) {
        Komputer k = new Komputer("Asus", "Intel", 3,
                                  "LG", "Logitech", "Wireless");
        k.displaySpec();
    }
}
```

Output:

```
Komputer: Asus
Processor: Intel, 3
Monitor: LG
Mouse: Logitech, Wireless
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  0.745 s
Finished at: 2026-10-19T20:02:10+07:00
------------------------------------------------------------------------
```

C. Unguided

Soal — Sistem Manajemen Pesanan Cafe

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.
Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (`Main`).

1. buatlah relasi komposisi melalui class RincianHarga dan ItemPesanan. Class RincianHarga memiliki field hargaDasar dan pajak yang bernilai 10% dari harga dasar, serta method getHargaTotal() yang mengembalikan jumlah keduanya. Class ItemPesanan memiliki field namaItem dan rincian yang bertipe RincianHarga, serta method getNamaItem() dan getRincian(). Objek RincianHarga harus dibuat langsung di dalam constructor ItemPesanan dan tidak boleh dibuat di main, sehingga jika ItemPesanan dihapus maka RincianHarga miliknya ikut hilang.

   Jawaban :

   RincianHarga

```
package Unguided;


public class RincianHarga {
```

```
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 10 / 100; // pajak 10%
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

ItemPesanan

```
package Unguided;

public class ItemPesanan {
    private String namaItem;
```

```
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        // Komposisi: RincianHarga dibuat di dalam constructor ItemPesanan
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
}
```

Penjelasan:
Class RincianHarga berfungsi menyimpan rincian harga dari satu item pesanan. Class ini memiliki dua field private, yaitu hargaDasar dan pajak, sehingga datanya tidak bisa diubah langsung dari luar class. Constructor menerima parameter hargaDasar yang namanya sama dengan field, maka this.hargaDasar = hargaDasar digunakan untuk membedakan field dengan parameter. Nilai pajak dihitung otomatis di constructor sebesar 10% dari harga dasar. Method getHargaDasar() dan getPajak() dipakai untuk membaca nilai field dari luar, sedangkan getHargaTotal() mengembalikan hasil penjumlahan harga dasar dan pajak.

Class ItemPesanan mewakili satu menu yang dipesan. Class ini memiliki field namaItem bertipe String dan rincian bertipe RincianHarga. Pada constructor, objek RincianHarga dibuat langsung dengan new RincianHarga(hargaDasar). Inilah yang membuat relasinya

berupa komposisi: RincianHarga tidak pernah dibuat di main dan siklus hidupnya terikat pada ItemPesanan, sehingga jika ItemPesanan dihapus maka RincianHarga miliknya ikut hilang. Method getNamaItem() dan getRincian() mengembalikan nama item dan objek rincian harga.

2. buatlah relasi agregasi melalui class Pesanan. Class ini memiliki field nomorMeja, daftarItem berupa array ItemPesanan[], dan jumlahItem sebagai penanda banyaknya item yang sudah masuk ke array. Method yang dibuat adalah tambahItem(ItemPesanan item), tampilkanPesanan(), getNomorMeja(), getDaftarItem(), dan getJumlahItem(). Objek ItemPesanan harus dibuat terlebih dahulu di main, lalu dimasukkan ke dalam Pesanan melalui tambahItem(), sehingga item tetap ada secara konsep walaupun pesanan dibatalkan.

   Jawaban :

   Pesanan

```
package Unguided;


public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;


    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }


    // Agregasi: ItemPesanan dibuat di luar, lalu dimasukkan ke sini
    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
```

```
        this.daftarItem[this.jumlahItem] = item;
        this.jumlahItem++;
    } else {
        System.out.println("Pesanan penuh, item tidak dapat ditambahkan.");
    }
}

public void tampilkanPesanan() {
    System.out.println("=== DAFTAR PESANAN CAFE ===");
    System.out.println("Nomor Meja: " + this.nomorMeja);
    for (int i = 0; i < this.jumlahItem; i++) {
        ItemPesanan item = this.daftarItem[i];
        RincianHarga r = item.getRincian();
        System.out.println((i + 1) + ". " + item.getNamaItem()
                + " - Rp " + r.getHargaTotal()
                + " (Harga: " + r.getHargaDasar()
                + ", Pajak: " + r.getPajak() + ")");
    }
}

public int getNomorMeja() {
    return this.nomorMeja;
}

public ItemPesanan[] getDaftarItem() {
    return this.daftarItem;
```

```
    }

    public int getJumlahItem() {
        return this.jumlahItem;
    }
}
```

Penjelasan:
Class Pesanan menampung kumpulan item yang dipesan pada satu meja. Field nomorMeja menyimpan nomor meja, daftarItem berupa array ItemPesanan[] untuk menampung item, dan jumlahItem menjadi penanda berapa item yang sudah masuk. Constructor membuat array dengan ukuran sesuai kapasitas dan mengisi jumlahItem dengan 0. Method tambahItem() memasukkan objek ItemPesanan ke posisi jumlahItem lalu menambahnya satu, dengan pengecekan agar program tidak error ketika array sudah penuh. Objek ItemPesanan dibuat di luar class (di main) lalu hanya dimasukkan ke sini, sehingga relasinya adalah agregasi. Method tampilkanPesanan() melakukan perulangan sebanyak jumlahItem untuk mencetak nomor meja serta nama, harga total, harga dasar, dan pajak tiap item. Method getter lainnya mengembalikan nomor meja, array item, dan jumlah item.

3. buatlah relasi asosiasi melalui class Kasir. Class ini hanya memiliki field namaKasir dan method prosesPembayaran(Pesanan pesanan) yang menghitung total bayar seluruh item lalu mencetak struk pembayaran. Class Kasir tidak boleh memiliki field bertipe Pesanan, karena objek Pesanan hanya diterima melalui parameter method.

   Jawaban :

   Kasir

```
package Unguided;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
```

```
        this.namaKasir = namaKasir;
    }

    // Asosiasi: Pesanan hanya diterima lewat parameter, tidak disimpan sebagai field
    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;
        ItemPesanan[] items = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            totalSubtotal += items[i].getRincian().getHargaDasar();
            totalPajak += items[i].getRincian().getPajak();
        }
        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

Penjelasan :
Class Kasir mewakili petugas yang memproses pembayaran. Class ini hanya memiliki satu field, yaitu namaKasir. Method prosesPembayaran(Pesanan pesanan) menerima objek Pesanan melalui parameter, bukan menyimpannya sebagai field, sehingga relasinya adalah asosiasi. Di dalam method, array item diambil lewat getDaftarItem(), lalu perulangan menjumlahkan harga dasar ke totalSubtotal dan pajak ke totalPajak dari setiap item. Total bayar adalah jumlah keduanya, kemudian struk dicetak lengkap dengan nama kasir, nomor meja, jumlah item, subtotal, pajak, dan total bayar.

4. buatlah class Main untuk menguji ketiga relasi tersebut. Mulai dengan membuat satu objek Kasir bernama “Budi”, kemudian buat tiga objek ItemPesanan secara independen, yaitu “Iced Americano” dengan harga dasar 18000, “Beef Croissant” dengan harga dasar 25000, dan “Matcha Latte” dengan harga dasar 22000. Selanjutnya buat satu objek Pesanan untuk Meja 4 dengan kapasitas array lima item, lalu masukkan ketiga item tadi menggunakan tambahItem(). Terakhir, tampilkan rincian pesanan dengan tampilkanPesanan() dan jalankan pembayaran dengan memanggil prosesPembayaran(pesanan) dari objek Kasir.

Jawaban :

Main

```
package Unguided;


public class Main {
    public static void main(String[] args) {
        Kasir kasir = new Kasir("Budi");


        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);


        Pesanan pesanan = new Pesanan(4, 5);
```

```
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);


        pesanan.tampilkanPesanan();
        System.out.println();
        kasir.prosesPembayaran(pesanan);
    }
}
```

Penjelasan :
Class Main menjalankan seluruh program lewat method main. Pertama dibuat objek Kasir bernama “Budi”, lalu tiga objek ItemPesanan dibuat secara terpisah dengan nama dan harga dasar masing-masing. Setelah itu dibuat objek Pesanan untuk Meja 4 dengan kapasitas lima item, dan ketiga item dimasukkan memakai tambahItem(). Terakhir, tampilkanPesanan() dipanggil untuk menampilkan daftar pesanan dan kasir.prosesPembayaran(pesanan) dipanggil untuk mencetak struk. Di sini terlihat jelas bahwa RincianHarga tidak pernah dibuat di main, sedangkan ItemPesanan dibuat di main lebih dulu sebelum dimasukkan ke Pesanan.

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.042 s
Finished at: 2026-10-19T20:00:07+07:00
------------------------------------------------------------------------
```

Penjelasan :
Output ini sama dengan output yang diharapkan di soal, dan tulisan BUILD SUCCESS menandakan program berjalan tanpa error.
Bagian Daftar Pesanan dicetak oleh tampilkanPesanan(). Program menampilkan nomor meja 4, lalu tiga item beserta harga total, harga dasar, dan pajak 10% masing-masing. Contohnya Iced Americano: harga dasar 18000.0 ditambah pajak 1800.0 menjadi 19800.0.
Bagian Struk Pembayaran dicetak oleh prosesPembayaran() milik Kasir. Struk menampilkan nama kasir (Budi), meja 4, dan jumlah item 3. Total subtotal 65000.0 (18000 + 25000 + 22000) ditambah total pajak 6500.0 menghasilkan total bayar 71500.0.

D. Kesimpulan

Dari tugas ini dapat disimpulkan bahwa relasi antar kelas dalam Java dapat diterapkan dengan tiga cara yang berbeda sesuai kuat lemahnya hubungan antar objek. Komposisi diterapkan pada ItemPesanan dan RincianHarga. RincianHarga dibuat di dalam constructor ItemPesanan sehingga siklus hidupnya terikat penuh pada objek induknya. Agregasi diterapkan pada Pesanan dan ItemPesanan. Objek ItemPesanan dibuat secara independen di main, lalu dimasukkan ke array daftarItem milik Pesanan,

sehingga item tetap dapat berdiri sendiri. Asosiasi diterapkan pada Kasir dan Pesanan. Kasir tidak menyimpan Pesanan sebagai field, tetapi hanya menerimanya lewat parameter method prosesPembayaran().
Program juga menerapkan enkapsulasi dengan field private yang diakses lewat getter, serta array standar untuk menampung data. Hasil program sesuai dengan output yang diharapkan, yaitu total bayar Rp 71500.0, sehingga ketiga relasi terbukti berjalan dengan benar.